

GPUMODE Lecture Study Notes: Lecture 3: Getting Started With CUDA for Python Programmers

Contents

1 Core Problem the Lecture Addresses

This lecture addresses a very practical problem:

How can a Python programmer, especially someone comfortable with PyTorch notebooks, start writing real CUDA kernels without first becoming an expert in C++, build systems, terminals, or low-level compiler tooling?

The lecture’s central claim is that CUDA is not as inaccessible as it seems. The difficulty comes less from the core programming model and more from the surrounding development friction: compiling code, moving between Python and C++, debugging kernels, and understanding how CUDA maps work onto GPU hardware.

The lecture builds a bridge from familiar Python and PyTorch code to CUDA by using the following workflow:

1. Write a clear, slow, correct Python implementation.
2. Rewrite the inner independent computation as a “kernel-like” Python function.
3. Simulate CUDA’s block and thread indexing in Python.
4. Convert the Python kernel to C++/CUDA syntax.
5. Use PyTorch’s inline extension system to compile and call the CUDA kernel from Python.
6. Compare correctness and performance against PyTorch implementations.

The lecture uses two core examples:

1. RGB image to grayscale conversion.
2. Naive matrix multiplication.

These examples are intentionally simple, but they expose almost every first concept needed to begin CUDA programming: linear memory, flattening, kernels, blocks, threads, guards, tensor pointers, CUDA launch syntax, one-dimensional and two-dimensional indexing, shared memory intuition, synchronization, and why library kernels are faster than naive kernels.

2 Required Background

2.1 PyTorch Tensors and Shape Conventions

The first example uses an RGB image loaded as a PyTorch tensor. In PyTorch, image tensors often use channel-first layout:

$$\text{image shape} = C \times H \times W,$$

where:

- C is the number of channels, usually 3 for RGB.
- H is the image height, or number of rows.

- W is the image width, or number of columns.

For the lecture's image, the original shape is described as approximately:

$$3 \times 1066 \times 1600.$$

This means there are three channel planes: red, green, and blue. Each channel contains 1066×1600 pixel values.

Matplotlib, however, expects color images in height-width-channel order:

$$H \times W \times C.$$

Therefore, when displaying an image loaded in PyTorch layout, the axes often need to be permuted:

$$(C, H, W) \mapsto (H, W, C).$$

```
def show_img(x, figsize=(4, 3), **kwargs):
    plt.figure(figsize=figsize)
    plt.axis("off")
    if len(x.shape) == 3:
        x = x.permute(1, 2, 0)
    plt.imshow(x.cpu(), **kwargs)
```

This function is not a CUDA concept by itself, but it establishes an important theme: before writing kernels, know the exact tensor layout.

2.2 Linear Memory Is the Real Representation

Although a tensor is viewed as multidimensional, GPU and CPU memory are linear. A tensor with shape

$$C \times H \times W$$

is ultimately stored as a one-dimensional sequence of values. In contiguous channel-first layout, all red pixels appear first, then all green pixels, then all blue pixels.

For an image with $N = HW$ pixels per channel, the flattened tensor is organized as:

$$\underbrace{R_0, R_1, \dots, R_{N-1}}_{\text{red channel}}, \underbrace{G_0, G_1, \dots, G_{N-1}}_{\text{green channel}}, \underbrace{B_0, B_1, \dots, B_{N-1}}_{\text{blue channel}}.$$

Therefore, for pixel index i , the red, green, and blue values are located at:

$$R_i = x[i],$$

$$G_i = x[i + N],$$

$$B_i = x[i + 2N].$$

This indexing pattern is crucial because simple CUDA kernels usually receive raw one-dimensional pointers, not high-level PyTorch tensor objects.

2.3 Contiguous Tensors

A tensor is contiguous if its logical ordering matches a compact linear memory layout. Many PyTorch operations, such as transpose, permute, narrow, or slicing, can create non-contiguous tensors.

The lecture emphasizes checking contiguity because a CUDA kernel that assumes flattened indexing will be wrong if the tensor is not contiguous.

The usual safety condition is:

kernel assumes flat indexing $\implies$ input tensor must be contiguous.

In PyTorch C++ extension code, this is often enforced using macros:

```
#define CHECK_CUDA(x) TORCH_CHECK(x.is_cuda(), #x " must be a CUDA tensor")
#define CHECK_CONTIGUOUS(x) TORCH_CHECK(x.is_contiguous(), #x " must be contiguous")
#define CHECK_INPUT(x) CHECK_CUDA(x); CHECK_CONTIGUOUS(x)
```

These checks prevent subtle memory-layout bugs.

2.4 CUDA Kernels

A CUDA kernel is a function that runs on the GPU many times in parallel. Conceptually, instead of writing a loop like this:

```
for i in range(n):
    out[i] = f(x, i)
```

CUDA asks the programmer to write only the body:

```
out[i] = f(x, i);
```

Then the CUDA runtime launches many threads, and each thread computes a different i .

A CUDA kernel normally does not return a value. Instead, it writes into output memory. This is because many parallel invocations run independently, and there is no single sequential return value.

CUDA kernel : many threads $\rightarrow$ write to output memory.

3 Main Concepts

3.1 Development Philosophy: Prototype in Python, Then Port to CUDA

The lecture strongly recommends the following workflow:

Write the computation slowly and clearly in Python first. Then isolate the independent inner computation and port that part to CUDA.

This is powerful because Python is easier to inspect, debug, print from, and reason about. A CUDA compile cycle can take tens of seconds, so debugging every idea directly in CUDA is painful.

The idea is not merely educational. It is a practical workflow:

1. Build a correct Python baseline.

2. Refactor the code to look like a CUDA kernel.
3. Simulate block and thread indexing in Python.
4. Verify correctness.
5. Convert to CUDA.
6. Compare output against the Python or PyTorch reference.

3.2 Example 1: RGB to Grayscale

3.2.1 Mathematical Formula

The lecture uses a standard luminance formula for converting RGB to grayscale:

$$Y = 0.21R + 0.72G + 0.07B.$$

Here:

- Y is the grayscale luminance value.
- R is the red channel value.
- G is the green channel value.
- B is the blue channel value.

The green channel receives the largest coefficient because human visual perception is more sensitive to green than to red or blue.

For a flattened channel-first image with $N = HW$, the formula becomes:

$$\text{out}[i] = 0.21 \cdot x[i] + 0.72 \cdot x[i + N] + 0.07 \cdot x[i + 2N].$$

This is the exact form that maps naturally to a CUDA thread.

3.2.2 Naive Python Implementation

A direct Python loop looks like this:

```
def rgb2gray_py(x):
    c, h, w = x.shape
    n = h * w
    x = x.flatten()
    out = torch.empty(n, dtype=x.dtype)

    for i in range(n):
        out[i] = 0.21 * x[i] + 0.72 * x[i + n] + 0.07 * x[i + 2 * n]

    return out.view(h, w)
```

The important indexing idea is that each output pixel only depends on three input values. There is no dependency between pixel i and pixel j . Therefore, each pixel can be computed independently.

This independence is exactly what makes the problem suitable for CUDA.

3.2.3 Parallelism in the Grayscale Kernel

The output has $N = HW$ values. A natural CUDA mapping is:

one CUDA thread $\leftrightarrow$ one output pixel.

Thread i computes:

$\text{out}[i]$.

Because all threads read separate locations and write separate output locations, there is no need for synchronization.

No cross-thread dependency $\implies$ no synchronization needed.

3.3 Python Simulation of a CUDA Kernel

Before writing CUDA, the lecture simulates the CUDA kernel launch model in Python.

A simple kernel runner can be written as:

```
def run_kernel(f, times, *args):  
    for i in range(times):  
        f(i, *args)
```

Then the kernel body can be separated from the loop:

```
def rgb2gray_kernel(i, out, x, n):  
    out[i] = 0.21 * x[i] + 0.72 * x[i + n] + 0.07 * x[i + 2 * n]
```

This approximates CUDA semantics: the kernel body does one independent unit of work, and a launcher calls it many times.

The actual CUDA version is not launched as a simple one-dimensional loop. CUDA organizes execution into blocks and threads.

3.4 Blocks and Threads

CUDA does not simply launch N independent scalar calls. Instead, it groups threads into thread blocks.

The core indexing formula is:

$$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

Here:

- blockIdx.x is the index of the current block.
- blockDim.x is the number of threads in each block.
- threadIdx.x is the index of the thread within the block.
- i is the global linear thread index.

If each block has T threads, then:

block 0 covers $i = 0, 1, \dots, T - 1$,

block 1 covers $i = T, T + 1, \dots, 2T - 1$,

block b covers $i = bT, \dots, (b + 1)T - 1$.

3.5 Choosing Threads Per Block

The lecture uses:

$$T = 256$$

as a reasonable default number of threads per block for beginner kernels.

If the problem has N independent items, the number of blocks is:

$$B = \left\lceil \frac{N}{T} \right\rceil.$$

A common integer ceiling division implementation is:

$$\left\lceil \frac{a}{b} \right\rceil = \left\lfloor \frac{a + b - 1}{b} \right\rfloor.$$

In C++:

```
inline int cdiv(int a, int b) {  
    return (a + b - 1) / b;  
}
```

3.6 The Guard Condition

Because N may not be divisible by T , the total launched thread count may exceed the number of valid output elements.

The launched thread count is:

$$BT = \left\lceil \frac{N}{T} \right\rceil T.$$

Usually:

$$BT \geq N.$$

Therefore, the kernel must guard against out-of-bounds indexing:

if $i < N$, do the work.

In CUDA code:

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
if (i < n) {
    out[i] = 0.21f * x[i] + 0.72f * x[i + n] + 0.07f * x[i + 2 * n];
}
```

This guard pattern appears in almost every beginner CUDA kernel.

4 Hardware-Level Explanation

4.1 Why GPUs Are Fast

The lecture contrasts CPU and GPU structure. CPUs are optimized for low-latency execution of relatively few complex instruction streams. GPUs are optimized for high-throughput execution of many similar operations across large amounts of data.

The lecture uses the RTX 3090 as an example GPU. It describes the card as having many streaming multiprocessors, or SMs, and many CUDA cores per SM. The exact numbers are less important than the architectural idea:

GPU throughput $\approx$ many SMs $\times$ many execution lanes per SM $\times$ many active threads.

An SM is the hardware unit that schedules and executes groups of GPU threads. A thread block is assigned to an SM, and all threads in that block execute on that SM.

4.2 Thread Blocks and SMs

The key hardware mapping is:

thread block $\rightarrow$ one SM.

A block does not get split across multiple SMs. This has important implications:

- Threads in the same block can cooperate.
- Threads in the same block can use shared memory.
- Threads in the same block can synchronize with each other.
- Threads in different blocks generally cannot synchronize inside one kernel.

This is why CUDA exposes blocks and threads instead of only exposing a single flat thread index. Blocks are not just a naming convenience; they define the unit of cooperation.

4.3 Shared Memory

Shared memory is a small, fast memory region local to an SM. Threads in the same block can read and write this shared memory.

shared memory = fast programmer-managed cache for one thread block.

The lecture emphasizes that shared memory is small but very fast. It is conceptually similar to a CPU cache, except the programmer explicitly decides what goes into it.

This matters because many kernels repeatedly reuse the same input values. If those values are loaded from slow global memory every time, performance suffers. If they are loaded once into shared memory and reused by all threads in a block, performance improves dramatically.

4.4 Global Memory Versus Shared Memory

Global memory is large but slow compared with shared memory and registers. Shared memory is small but fast. Registers are even closer to the execution units and are private to each thread.

A simplified hierarchy is:

registers $\rightarrow$ shared memory $\rightarrow$ L1/L2 cache $\rightarrow$ global memory.

The performance intuition is:

more reuse from registers/shared memory $\implies$ less global memory traffic $\implies$ higher performance.

4.5 Synchronization

Threads in the same block can synchronize. The conceptual operation is:

all threads in block must reach this point before any continue.

In CUDA C++, this is usually written as:

```
__syncthreads();
```

The lecture does not build a shared-memory tiled matrix multiplication kernel, but it explains that future optimized kernels use block-level cooperation and synchronization to reuse data.

4.6 Registers

The lecture briefly notes that GPUs have substantial register memory, often even more total register capacity than shared memory. Registers are private to each thread. They are extremely fast, but excessive register use can reduce occupancy because the SM has a fixed register file shared among resident threads.

The trade-off is:

more registers per thread $\implies$ fewer resident threads per SM $\implies$ possibly lower occupancy.

5 Mathematical Reasoning

5.1 RGB to Grayscale Indexing

For an image in contiguous $C \times H \times W$ format:

$$N = HW.$$

The flattened address for channel c , row r , and column q is:

$$\text{idx}(c, r, q) = cHW + rW + q.$$

If we flatten the pixel coordinate (r, q) into:

$$i = rW + q,$$

then:

$$\text{red index} = i,$$

$$\text{green index} = i + N,$$

$$\text{blue index} = i + 2N.$$

The grayscale output is:

$$Y_i = 0.21x_i + 0.72x_{i+N} + 0.07x_{i+2N}.$$

The total arithmetic per pixel is approximately:

$$3 \text{ multiplications} + 2 \text{ additions}.$$

The total work is:

$$O(HW).$$

Because each pixel is independent, the available parallelism is:

$$P = HW.$$

For the resized example:

$$H = 150, \quad W = 225,$$

$$P = 150 \cdot 225 = 33750.$$

For the full image:

$$H = 1066, \quad W = 1600,$$

$$P = 1066 \cdot 1600 = 1705600.$$

This is enough independent work to benefit substantially from GPU parallelism.

5.2 Kernel Launch Geometry

Let:

N = number of output elements,

T = threads per block,

$$B = \left\lceil \frac{N}{T} \right\rceil.$$

The total number of launched threads is:

$$N_{\text{launched}} = BT.$$

The number of excess threads is:

$$N_{\text{excess}} = BT - N.$$

The guard condition eliminates invalid work:

$$i < N.$$

5.3 Matrix Multiplication Formula

For matrix multiplication:

$$C = AB,$$

where:

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N}.$$

Each output element is:

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} B_{kj}.$$

The shapes must satisfy:

$$A_{\text{cols}} = B_{\text{rows}} = K.$$

The output shape is:

$$M \times N.$$

The total operation count is approximately:

$$2MNK$$

floating point operations if each multiply and add is counted separately.

In the lecture's MNIST example, the images are 28×28 , so each flattened image has:

$$28 \cdot 28 = 784$$

features. A linear classifier for 10 digits uses a weight matrix:

$$W \in \mathbb{R}^{784 \times 10}.$$

If the input batch has M images:

$$X \in \mathbb{R}^{M \times 784},$$

then:

$$Y = XW, \quad Y \in \mathbb{R}^{M \times 10}.$$

5.4 Naive Matrix Multiplication Parallelization

A simple CUDA mapping is:

one CUDA thread $\leftrightarrow$ one output element C_{ij} .

Each thread computes one dot product:

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} B_{kj}.$$

This exposes:

$$MN$$

parallel output elements. However, each thread still performs K serial multiply-add operations. The mapping is simple and correct, but it does not fully exploit reuse.

5.5 Arithmetic Intensity

Arithmetic intensity is:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes moved}}.$$

A high arithmetic intensity means each byte loaded from memory contributes to many arithmetic operations. Such kernels are more likely to be compute-bound.

A low arithmetic intensity means the GPU spends much of its time moving data rather than computing. Such kernels are more likely to be memory-bound.

For RGB to grayscale, each output pixel reads three input bytes and writes one output byte, ignoring type conversion details:

$$\text{bytes moved per pixel} \approx 3 + 1 = 4.$$

The arithmetic is approximately 5 scalar operations per pixel. Thus:

$$\text{AI}_{\text{grayscale}} \approx \frac{5}{4}.$$

This is low. The kernel is simple and likely memory-bandwidth-sensitive.
For matrix multiplication, each output element performs K multiply-adds:

$$\text{FLOPs per output} \approx 2K.$$

Naively, each output element reads K values from A , K values from B , and writes one value to C . For FP32 values:

$$\text{bytes per output} \approx 4K + 4K + 4 = 8K + 4.$$

Thus the naive arithmetic intensity per output is roughly:

$$\text{AI}_{\text{naive matmul}} \approx \frac{2K}{8K + 4}.$$

For large K , this approaches:

$$\frac{2K}{8K} = \frac{1}{4}.$$

This looks poor because it ignores reuse. Optimized matrix multiplication improves arithmetic intensity by reusing tiles of A and B from shared memory and registers.

6 Code Walkthrough

6.1 Notebook Setup for CUDA Extensions

The lecture uses notebooks, including Google Colab, rather than a separate CMake or terminal workflow. PyTorch provides inline extension compilation through:

```
from torch.utils.cpp_extension import load_inline
```

A convenient wrapper can compile strings of C++ and CUDA code into a Python-importable module.

```
def load_cuda(cuda_src, cpp_src, funcs, opt=False, verbose=False):
    return load_inline(
        cuda_sources=[cuda_src],
        cpp_sources=[cpp_src],
        functions=funcs,
        extra_cuda_cflags=["-O2"] if opt else [],
        verbose=verbose,
        name="inline_ext"
    )
```

The key idea is that CUDA source code is passed as a string. PyTorch builds the extension, links it, loads it into Python, and exposes selected functions.

6.2 Debugging Setup

The lecture recommends enabling synchronous CUDA error reporting while developing:

```
import os
os.environ["CUDA_LAUNCH_BLOCKING"] = "1"
```

Normally, CUDA launches are asynchronous: Python may continue executing before the GPU work is complete. This is good for performance but bad for debugging. With launch blocking enabled, errors are reported closer to the line that caused them.

The lecture also recommends using `wurlitzer` in notebooks so that C++ or CUDA print output can appear in the notebook.

```
%load_ext wurlitzer
```

This is especially useful when debugging small kernels.

6.3 Common C++ Header and Utility Code

The lecture uses a shared C++ source string containing includes, input checks, and ceiling division.

```
#include <torch/extension.h>
#include <iostream>
#include <stdexcept>

#define CHECK_CUDA(x) TORCH_CHECK(x.is_cuda(), #x " must be a CUDA tensor")
#define CHECK_CONTIGUOUS(x) TORCH_CHECK(x.is_contiguous(), #x " must be contiguous")
#define CHECK_INPUT(x) CHECK_CUDA(x); CHECK_CONTIGUOUS(x)

inline int cdiv(int a, int b) {
    return (a + b - 1) / b;
}
```

Line-by-line meaning:

- `torch/extension.h` exposes PyTorch tensor APIs to C++.
- `CHECK_CUDA` ensures the tensor lives on the GPU.
- `CHECK_CONTIGUOUS` ensures flattened pointer indexing is valid.
- `CHECK_INPUT` combines both checks.
- `cdiv` computes the number of blocks needed for a given number of elements.

6.4 CUDA RGB to Grayscale Kernel

A CUDA version of the grayscale kernel is:

```
__global__ void rgb_to_grayscale_kernel(
    unsigned char* out,
    unsigned char* x,
    int n
) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < n) {
        out[i] = 0.21f * x[i]
            + 0.72f * x[i + n]
            + 0.07f * x[i + 2 * n];
    }
}
```

Important details:

- `__global__` means this function is a CUDA kernel callable from host code and executed on the GPU.
- `unsigned char*` corresponds to byte-like image storage.
- The pointer `x` is treated as a one-dimensional array.
- `blockIdx.x`, `blockDim.x`, and `threadIdx.x` produce the global pixel index.
- The guard `if (i < n)` prevents out-of-bounds memory access.

6.5 Calling the RGB Kernel from C++

The host-side C++ function allocates the output tensor, launches the kernel, checks for errors, and returns the result.

```
torch::Tensor rgb_to_grayscale(torch::Tensor x) {
    CHECK_INPUT(x);

    int c = x.size(0);
    int h = x.size(1);
    int w = x.size(2);
    int n = h * w;

    auto out = torch::empty({h, w}, x.options());

    int threads = 256;
    int blocks = cdiv(n, threads);

    rgb_to_grayscale_kernel<<<blocks, threads>>>(
        out.data_ptr<unsigned char>(),
        x.data_ptr<unsigned char>(),
        n
    );

    C10_CUDA_KERNEL_LAUNCH_CHECK();

    return out;
}
```

Key C++ and CUDA concepts:

- `torch::Tensor` is the C++ type for PyTorch tensors.
- `x.size(1)` accesses tensor dimensions.
- `torch::empty({h, w}, x.options())` creates an output tensor with the same dtype and device as `x`.
- `data_ptr<unsigned char>()` extracts a raw pointer of the requested type.
- The triple angle bracket syntax launches the kernel.
- `C10_CUDA_KERNEL_LAUNCH_CHECK()` checks whether the kernel launch failed.

6.6 Header Declaration for Exposed Function

The C++ source string also needs a function declaration:

```
torch::Tensor rgb_to_grayscale(torch::Tensor x);
```

This tells PyTorch's extension machinery which function exists and can be made callable from Python.

6.7 Python Use of the Compiled Module

Once compiled, the function can be called from Python:

```
imgc = img.contiguous().cuda()  
gray = module.rgb_to_grayscale(imgc)  
show_img(gray, cmap="gray")
```

The important preparation steps are:

```
contiguous() and cuda().
```

The first ensures layout correctness. The second places the tensor on GPU memory.

6.8 CUDA Asynchrony and Timing

CUDA launches are usually asynchronous. This means the CPU can continue running while the GPU is still executing the kernel.

Therefore, this timing can be misleading:

```
%time y = module.rgb_to_grayscale(imgc)
```

The measured time may only include the launch overhead, not the full execution. To force synchronization, one can move the result back to CPU:

```
%time y = module.rgb_to_grayscale(imgc).cpu()
```

or explicitly synchronize:

```
torch.cuda.synchronize()
```

The lecture includes the CPU transfer in timing to ensure the CUDA work has completed.

6.9 Naive Python Matrix Multiplication

The lecture next implements matrix multiplication manually.

```
def matmul_py(a, b):  
    ar, ac = a.shape  
    br, bc = b.shape  
    assert ac == br  
  
    c = torch.zeros(ar, bc)  
  
    for i in range(ar):  
        for j in range(bc):  
            for k in range(ac):
```



```

        c[i, j] += a[i, k] * b[k, j]

    return c

```

This is a direct implementation of:

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} B_{kj}.$$

It is clear but slow because it uses Python loops. Python loop overhead dominates.

6.10 Kernel-Like Python Matrix Multiplication

The CUDA mapping assigns one output element to one thread.

```

def matmul_kernel(row, col, a, b, c, h, w, k):
    if row < h and col < w:
        acc = 0.0
        for i in range(k):
            acc += a[row * k + i] * b[i * w + col]
        c[row * w + col] = acc

```

This version already resembles CUDA code. The output element is addressed by (row, col), and the dot product is serial inside one thread.

6.11 Two-Dimensional Blocks and Threads

CUDA block and thread indices can be one-dimensional, two-dimensional, or three-dimensional. For matrix multiplication, a two-dimensional launch is convenient.

The row and column are computed as:

$$\text{row} = \text{blockIdx.y} \cdot \text{blockDim.y} + \text{threadIdx.y},$$

$$\text{col} = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

The kernel guard becomes:

$$\text{row} < M \quad \text{and} \quad \text{col} < N.$$

6.12 CUDA Matrix Multiplication Kernel

A direct CUDA kernel is:

```

__global__ void matmul_kernel(
    float* a,
    float* b,
    float* c,
    int h,
    int w,
    int k
) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

```

```

    if (row < h && col < w) {
        float acc = 0.0f;

        for (int i = 0; i < k; ++i) {
            acc += a[row * k + i] * b[i * w + col];
        }

        c[row * w + col] = acc;
    }
}

```

Line-by-line hardware interpretation:

- Each thread computes one output cell.
- Threads in the same block cover a tile of the output matrix.
- The loop over i is serial inside each thread.
- The access to $a[\text{row} * k + i]$ walks contiguously across a row of A .
- The access to $b[i * w + \text{col}]$ walks down a column of B , which is less cache-friendly in row-major memory.
- The output write $c[\text{row} * w + \text{col}]$ is one write per thread.

6.13 CUDA Matrix Multiplication Host Function

```

torch::Tensor matmul(torch::Tensor a, torch::Tensor b) {
    CHECK_INPUT(a);
    CHECK_INPUT(b);

    int h = a.size(0);
    int k = a.size(1);
    int k2 = b.size(0);
    int w = b.size(1);

    TORCH_CHECK(k == k2, "Inner dimensions must match");

    auto c = torch::zeros({h, w}, a.options());

    dim3 tpb(16, 16);
    dim3 blocks(cdiv(w, tpb.x), cdiv(h, tpb.y));

    matmul_kernel<<<blocks, tpb>>>(
        a.data_ptr<float>(),
        b.data_ptr<float>(),
        c.data_ptr<float>(),
        h,
        w,
        k
    );
}

```

```
C10_CUDA_KERNEL_LAUNCH_CHECK();  
  
    return c;  
}
```

Important details:

- `dim3 tpb(16, 16)` creates a two-dimensional block of $16 \times 16 = 256$ threads.
- `blocks.x` covers output columns.
- `blocks.y` covers output rows.
- The output has shape $h \times w$.
- `TORCH_CHECK` validates that A 's columns match B 's rows.

6.14 Why 16×16 Threads Per Block?

A 16×16 thread block contains:

$$16 \cdot 16 = 256$$

threads. This matches the lecture's earlier default of 256 threads per block while making the two-dimensional output mapping convenient.

The shape of the block is chosen for programmer convenience. CUDA only requires that the total number of threads per block remain within the hardware limit. A common maximum is 1024 threads per block, although exact limits depend on hardware.

7 Performance Intuition

7.1 Why the Python Version Is Slow

The naive Python grayscale loop is slow because every pixel runs through Python interpreter overhead.

For N pixels, Python executes N loop iterations, and each iteration performs multiple Python-level operations:

$$\text{time} \approx N \cdot (\text{Python loop overhead} + \text{tensor indexing overhead} + \text{arithmetic}).$$

For only about 33750 pixels, this can already take seconds.

The GPU version moves the loop into parallel compiled code:

Python launches once $\rightarrow$ GPU runs many threads.

This removes Python from the inner loop and exposes massive parallelism.

7.2 Why the Grayscale CUDA Kernel Is Fast

The grayscale kernel is fast because:

- There is one independent thread per pixel.
- The computation is simple.
- The memory accesses are predictable.
- There is no synchronization.
- There are no atomics.
- There is no inter-thread communication.

However, it is not compute-heavy. It mostly moves data. Therefore, it is likely bandwidth-bound.

few arithmetic operations per byte $\implies$ memory-bandwidth-sensitive.

7.3 Why the Naive CUDA Matrix Multiplication Is Faster Than Python

The naive CUDA matrix multiplication is much faster than Python because:

- Many output cells are computed in parallel.
- The inner dot-product loop is compiled C++/CUDA code.
- Python loop overhead is eliminated.
- The GPU has many execution lanes for parallel work.

The lecture reports a large improvement over a CPU/Python-style implementation.

7.4 Why PyTorch Matrix Multiplication Is Still Faster

PyTorch matrix multiplication calls highly optimized vendor libraries such as cuBLAS. These kernels are much faster than naive one-thread-per-output matmul because they use deeper optimization strategies:

- Tiling.
- Shared memory reuse.
- Register blocking.
- Vectorized memory access.
- Memory coalescing.
- Tensor cores for FP16, BF16, TF32, or other supported formats.
- Instruction scheduling optimized for the GPU architecture.

The naive kernel repeatedly loads values from global memory. For example, many neighboring output elements reuse the same rows of A and columns of B , but the naive kernel does not exploit that reuse.

7.5 Tiling Intuition for Matrix Multiplication

Consider a block of threads computing a tile of C . The same tile of A and the same tile of B are reused by many threads.

Instead of every thread reading directly from global memory independently, an optimized kernel can do:

1. Load a tile of A into shared memory.
2. Load a tile of B into shared memory.
3. Synchronize the block.
4. Have many threads reuse the shared-memory tiles.
5. Accumulate partial sums in registers.
6. Move to the next tile along the K dimension.

The tiled algorithm increases arithmetic intensity because each global memory load supports many multiply-add operations.

global load once $\rightarrow$ reuse many times from shared memory.

7.6 Memory Coalescing

For good global memory throughput, adjacent threads in a warp should ideally access adjacent memory addresses.

In the grayscale kernel, adjacent threads with indices i , $i + 1$, $i + 2$, and so on read adjacent red values:

$$x[i], x[i + 1], x[i + 2], \dots$$

They also read adjacent green and blue values:

$$x[i + N], x[i + 1 + N], x[i + 2 + N], \dots$$

This is favorable for coalescing.

In naive matrix multiplication, memory access is mixed:

- Accesses to rows of A are contiguous inside each thread's loop.
- Accesses to columns of B are strided for each thread.
- Across neighboring threads computing neighboring columns, accesses to B can be more favorable for a fixed i .

Optimized kernels carefully organize thread mappings and memory layouts to maximize coalesced global loads.

7.7 Kernel Launch Overhead

A CUDA kernel launch has overhead. If the computation is tiny, launch overhead may dominate. Therefore, CUDA is most useful when the amount of parallel work is large enough.

small problem $\implies$ launch overhead can dominate.

large problem $\implies$ parallel GPU execution dominates.

This is why the lecture resizes the image for slow Python experimentation but uses the full image for CUDA timing.

7.8 Asynchronous Execution

CUDA kernels usually launch asynchronously. This means the CPU does not automatically wait for the GPU kernel to finish.

A common benchmarking mistake is:

```
start = time.time()
y = module.my_kernel(x)
end = time.time()
```

This may only measure launch time. Correct timing needs synchronization:

```
start = time.time()
y = module.my_kernel(x)
torch.cuda.synchronize()
end = time.time()
```

or a forced transfer:

```
y_cpu = module.my_kernel(x).cpu()
```

8 One-Dimensional Versus Two-Dimensional Launches

8.1 One-Dimensional Launch

A one-dimensional launch is often simplest:

$$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

It is natural for flattened arrays. RGB to grayscale works cleanly in one dimension because the output is just N independent pixels.

8.2 Two-Dimensional Launch

A two-dimensional launch is often convenient for matrix-like outputs:

$$\text{row} = \text{blockIdx.y} \cdot \text{blockDim.y} + \text{threadIdx.y},$$

$$\text{col} = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

This makes the code match the mathematical structure of a matrix.

8.3 Two-Dimensional RGB to Grayscale

The lecture also shows that RGB to grayscale can be written using two-dimensional blocks, but that does not necessarily make it simpler.

A two-dimensional grayscale kernel must compute both row and column, then convert back to a flattened pixel index:

$$i = \text{row} \cdot W + \text{col}.$$

The guard must check both dimensions:

$$\text{row} < H \quad \text{and} \quad \text{col} < W.$$

For this problem, one-dimensional indexing is shorter and clearer. The main lesson is:

Use one-dimensional, two-dimensional, or three-dimensional launches only when they make the mapping clearer. They are conveniences, not requirements.

9 CUDA Extension Concepts

9.1 `__global__`, `__device__`, and `__host__`

CUDA extends C++ with function qualifiers.

- `__global__`: callable from CPU host code and runs on the GPU.
- `__device__`: callable from GPU code and runs on the GPU.
- `__host__`: callable from CPU code and runs on the CPU. This is the default for normal C++ functions.

Most beginner kernels use `__global__` because Python or C++ host code launches a GPU kernel.

9.2 The Triple Angle Bracket Syntax

CUDA kernel launches use special syntax:

```
kernel<<<blocks, threads>>>(arguments);
```

For example:

```
rgb_to_grayscale_kernel<<<blocks, threads>>>(out_ptr, x_ptr, n);
```

The values inside the triple angle brackets describe launch geometry, not normal function arguments.

9.3 dim3

CUDA uses `dim3` to represent one-dimensional, two-dimensional, or three-dimensional launch geometry.

```
dim3 tpb(16, 16);
dim3 blocks(cdiv(w, tpb.x), cdiv(h, tpb.y));
```

If only one or two dimensions are specified, the unspecified dimensions default to 1. Therefore:

`dim3(256)`

is equivalent to:

`(256, 1, 1)`.

9.4 Raw Pointers from PyTorch Tensors

CUDA kernels usually receive raw pointers:

```
x.data_ptr<float>()
```

This converts a PyTorch tensor into a pointer that CUDA C++ can index.

For FP32 tensors:

`float*`

is used.

For unsigned 8-bit image tensors:

`unsigned char*`

is used.

The pointer type must match the tensor dtype. If the dtype is wrong, the kernel will reinterpret bytes incorrectly.

9.5 Why Kernels Write to Output Memory

CUDA kernels usually return `void`. They communicate results by writing into memory passed as a pointer.

This is because thousands or millions of kernel instances are executing. There is no single return value from all threads.

The pattern is:

```
__global__ void kernel(float* out, float* x, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        out[i] = x[i] * x[i];
    }
}
```


10 Common Misconceptions

10.1 Misconception 1: CUDA Requires Leaving Notebooks

CUDA can be used inside notebooks through PyTorch’s inline extension machinery. A notebook is not only acceptable for learning; it can be a productive environment for iterative CUDA development.

10.2 Misconception 2: CUDA Kernels Return Values Like Python Functions

A CUDA kernel usually does not return a scalar or tensor directly. It writes to output memory. A host-side wrapper can allocate a tensor, pass its pointer to the kernel, and return that tensor to Python.

10.3 Misconception 3: Blocks and Threads Are Arbitrary Syntax

Blocks and threads correspond to hardware organization. Threads in the same block execute on the same SM and can share memory and synchronize. This is the reason CUDA exposes the block hierarchy.

10.4 Misconception 4: Two-Dimensional Data Requires Two-Dimensional CUDA Blocks

Two-dimensional blocks are optional. A two-dimensional image or matrix can be processed with one-dimensional indexing. Use two-dimensional launches when they make the indexing easier or the hardware mapping better.

10.5 Misconception 5: A Naive CUDA Kernel Should Match PyTorch Performance

Naive CUDA code can be much faster than Python but still slower than PyTorch. PyTorch calls highly optimized kernels that use shared memory, registers, tiling, vectorization, and sometimes tensor cores.

10.6 Misconception 6: GPU Timing Is Straightforward

CUDA execution is asynchronous. Timing a kernel without synchronization can severely under-report execution time. Correct benchmarking requires synchronization or a CPU transfer.

10.7 Misconception 7: Flattening Is Just a Convenience

Flattening is not merely a convenience. It reflects how memory is actually addressed. CUDA kernels often operate on raw one-dimensional pointers, so understanding flattened indexing is essential.

10.8 Misconception 8: Shared Memory Is Automatic Cache

Shared memory is programmer-managed. It is not the same as an automatic CPU cache. The programmer explicitly loads values into shared memory, synchronizes, and reuses them.

11 Practical Takeaways

1. Start with a correct Python implementation before writing CUDA.
2. Refactor the independent inner computation into a kernel-like function.
3. Think in terms of one thread computing one independent output element.
4. Use flattened indexing until multidimensional indexing is truly helpful.
5. Always include guard conditions for out-of-bounds threads.
6. Use 256 threads per block as a reasonable beginner default.
7. Use `dim3(16, 16)` for simple two-dimensional matrix-style kernels.
8. Always check that tensors are CUDA tensors and contiguous.
9. Always check kernel launch errors during development.
10. Remember that CUDA launches are asynchronous when benchmarking.
11. Expect naive CUDA to beat Python but not optimized PyTorch.
12. Shared memory and register tiling are the next step after naive kernels.

12 Project or Experiment Ideas

12.1 Experiment 1: Grayscale Kernel Variants

Implement RGB to grayscale in three ways:

1. Pure Python loop.
2. PyTorch vectorized expression.
3. Custom CUDA kernel.

Measure:

latency, effective bandwidth, correctness error.

Effective bandwidth can be estimated as:

$$\text{Bandwidth} = \frac{\text{bytes read} + \text{bytes written}}{\text{time}}.$$

12.2 Experiment 2: One-Dimensional Versus Two-Dimensional Image Kernels

Write two grayscale kernels:

- One-dimensional flattened kernel.
- Two-dimensional row-column kernel.

Compare code clarity and performance. The important lesson is not that one form is always faster, but that indexing choices affect both readability and memory behavior.

12.3 Experiment 3: Naive Matrix Multiplication

Implement:

1. Python triple-loop matrix multiplication.
2. Python broadcasting matrix multiplication.
3. Naive CUDA matrix multiplication.
4. PyTorch @ operator.

Measure speed for different shapes:

$$(M, K, N) \in \{(128, 128, 128), (512, 512, 512), (1024, 1024, 1024)\}.$$

12.4 Experiment 4: Add Shared Memory Tiling

Upgrade the naive CUDA matrix multiplication kernel to use shared memory. Use a tile size such as:

$$T = 16.$$

Each block computes a 16×16 tile of C . For each tile of the K -dimension:

1. Load a tile of A into shared memory.
2. Load a tile of B into shared memory.
3. Synchronize.
4. Accumulate partial sums.
5. Synchronize before loading the next tile.

Compare against the naive kernel and PyTorch.

12.5 Experiment 5: Benchmark With Correct Synchronization

Write a timing helper:

```
def time_cuda(fn, *args, warmup=10, iters=100):
    for _ in range(warmup):
        y = fn(*args)
        torch.cuda.synchronize()

    start = time.time()
    for _ in range(iters):
        y = fn(*args)
        torch.cuda.synchronize()

    return (time.time() - start) / iters
```

Use it to compare all kernels. This experiment teaches the importance of CUDA synchronization.

12.6 Experiment 6: Implement a Simple Fused Kernel

Try a fused operation such as:

$$y_i = \text{ReLU}(ax_i + b).$$

Compare:

```
y = torch.relu(a * x + b)
```

against a custom CUDA kernel that performs the multiply, add, and ReLU in one pass over memory.

This connects directly to why CUDA matters in deep learning systems: fusion reduces memory traffic.

12.7 Experiment 7: Read Real CUDA Code

Read small parts of open-source implementations of:

- FlashAttention.
- bitsandbytes.
- GPTQ kernels.
- AWQ kernels.

Do not try to understand everything at once. Identify:

1. The kernel launch geometry.
2. The tensor pointer types.
3. The guard conditions.
4. Shared memory usage.
5. Synchronization points.
6. Accumulation dtype.

13 Visual Concept

Visual Concept: Python Loop to CUDA Kernel

Draw a left-to-right diagram. On the left, show a Python `for` loop iterating over pixels one at a time. On the right, show many CUDA threads each receiving a different index i and writing one output pixel.

Visual Concept: Flattened RGB Layout

Draw three stacked channel planes labeled R , G , and B . Then draw them flattened into one long vector: all red values first, then all green values, then all blue values. Mark $x[i]$, $x[i + N]$, and $x[i + 2N]$.

Visual Concept: Blocks and Threads

Draw a grid of blocks. Inside one block, draw a smaller grid of threads. Label `blockIdx.x`, `threadIdx.x`, and `blockDim.x`. Show the formula $i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$.

Visual Concept: Matrix Multiplication Thread Mapping

Draw matrix C divided into 16×16 tiles. Show one CUDA block assigned to one tile. Inside the tile, each thread computes one C_{ij} by taking a dot product between a row of A and a column of B .

Visual Concept: Shared Memory Tiling

Draw global memory feeding tiles of A and B into shared memory inside an SM. Then draw many threads reading from shared memory repeatedly and accumulating values in registers before writing a tile of C back to global memory.

14 Hardware-Level Summary of the Two Examples

14.1 RGB to Grayscale

one thread $\rightarrow$ one pixel.

Properties:

- Very simple indexing.
- Coalesced reads are likely for contiguous channel-first data.
- No thread cooperation.
- No shared memory needed.
- Low arithmetic intensity.
- Likely bandwidth-bound.

14.2 Naive Matrix Multiplication

one thread $\rightarrow$ one output matrix element.

Properties:

- Simple two-dimensional indexing.
- Each thread performs a serial dot product.
- Substantial repeated global memory reads.
- No shared memory reuse.
- Faster than Python but slower than optimized PyTorch.
- Natural next step is tiled shared-memory matmul.

15 Connection to Deep Learning Systems

The lecture ends by motivating why CUDA knowledge matters for modern deep learning. Many increasingly important model optimizations cannot be expressed efficiently in ordinary PyTorch alone.

Examples include:

- FlashAttention.
- Quantization kernels.
- GPTQ.
- AWQ.
- bitsandbytes.
- Fused activation and normalization kernels.
- Custom memory-efficient attention variants.

The reason is that PyTorch operations are often too coarse or too general. A model-specific optimization may require controlling:

- which values are loaded,
- how they are laid out in memory,
- how intermediate results are accumulated,
- whether intermediate tensors are materialized,
- whether operations are fused,
- whether shared memory or registers are used,
- how precision is managed.

A PyTorch expression might be mathematically concise but create unnecessary memory traffic. A custom CUDA kernel can fuse operations and avoid materializing intermediate tensors.

16 Local Development Setup

The lecture also discusses setting up CUDA development outside Colab.

The recommended approach is to use Conda as an environment manager. The key idea is that Conda can manage not only Python packages, but also CUDA versions, compilers, and binary dependencies.

The workflow is:

1. Install Conda or Miniconda.
2. Identify the CUDA version expected by the current PyTorch installation.

3. Install the matching NVIDIA CUDA toolkit packages through Conda.
4. Install PyTorch with CUDA support.

The lecture frames Conda less as a replacement for `pip` and more as a lightweight alternative to Docker for managing binary environments.

This is useful because CUDA development often depends on exact compatibility among:

- NVIDIA driver version.
- CUDA toolkit version.
- PyTorch version.
- C++ compiler version.
- Operating system environment.

The lecture notes that CUDA development works naturally on Linux and Windows through WSL. Native macOS CUDA development is not applicable because modern Macs do not support NVIDIA CUDA GPUs.

17 Common Kernel Design Pattern

A beginner CUDA kernel usually follows this pattern:

1. Compute a global thread index.
2. Check a guard condition.
3. Load input values from global memory.
4. Perform arithmetic.
5. Store the result to global memory.

For one-dimensional data:

```
__global__ void kernel(float* out, float* x, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < n) {
        out[i] = x[i] * x[i];
    }
}
```

For two-dimensional data:

```
__global__ void kernel2d(float* out, float* x, int h, int w) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < h && col < w) {
        int i = row * w + col;
        out[i] = x[i] * x[i];
    }
}
```

This pattern is the foundation for more advanced kernels.

18 Final Mental Model

The lecture's final mental model is:

A CUDA kernel is the body of a loop that the GPU runs many times in parallel. Each thread receives an identity through `blockIdx`, `blockDim`, and `threadIdx`, uses that identity to decide which data element it owns, checks that it is in bounds, and writes its result to memory. Python remains the orchestration layer, PyTorch provides tensor allocation and compilation machinery, and CUDA provides the parallel execution model.

For RGB to grayscale:

$$\text{thread } i \rightarrow 0.21R_i + 0.72G_i + 0.07B_i.$$

For matrix multiplication:

$$\text{thread } (i, j) \rightarrow \sum_{k=0}^{K-1} A_{ik}B_{kj}.$$

The beginner path is:

Python loop $\rightarrow$ kernel-like Python $\rightarrow$ CUDA kernel $\rightarrow$ optimized CUDA kernel.

The deeper systems lesson is:

Correct CUDA starts with indexing. Fast CUDA starts with memory reuse.